

NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Design a Distributed Application Using RPC for Remote Computation of Factorial

Code:

```
#server.py

from xmlrpc.server import SimpleXMLRPCServer
# Function to calculate factorial
def factorial(n):
    if n < 0:
        return "Factorial not defined for negative numbers"
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
# Create RPC server
server = SimpleXMLRPCServer(("localhost", 8000))
print("RPC Server is running on port 8000...")
# Register function
server.register_function(factorial, "factorial")

# Run server
server.serve_forever()

#client.py
import xmlrpc.client
# Connect to RPC server
server = xmlrpc.client.ServerProxy("http://localhost:8000/")
# Take input from user
```

```
num = int(input("Enter a number: "))  
# Call remote factorial function  
result = server.factorial(num)  
# Display result  
print("Factorial of", num, "is:", result)
```

Output:

RPC Server is running on port 8000...

Enter a number: 5

Factorial of 5 is: 120

NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Design a Distributed Application Using RMI for Remote String Concatenation.

Code:

#server.py

```
import Pyro4
@Pyro4.expose
class StringConcatenator:
    def concatenate(self, s1, s2):
        return s1 + s2
# Start Pyro daemon
daemon = Pyro4.Daemon()
uri = daemon.register(StringConcatenator)
print("Server is ready.")
print("URI:", uri)
daemon.requestLoop()
```

#client.py

```
import Pyro4
# Get the server object using its URI
uri = input("Enter the server URI : ")
string_concatenator = Pyro4.Proxy(uri)
# Get inputs string from the user
str1 = input("Enter first string : ")
str2 = input("Enter second string : ")
```

```
# Call the remote method on the server
result = string_concatenator.concatenate(str1, str2)
print("Concatenated String :", result)
```

Output:

Server is ready.

URI: PYRO:obj_6faec84451ed453caca91c47aabb465@localhost:58607

Enter the server URI : PYRO:obj_6faec84451ed453caca91c47aabb465@localhost:58607

Enter first string : Swami

Enter second string : Aher

Concatenated String : SwamiAher

NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Implementation of Fuzzy Set Operations and Fuzzy Relations.

Code:

```
# -----  
# Fuzzy Set Operations Functions  
# -----  
def fuzzy_union(A, B):  
    return {x: max(A[x], B[x]) for x in A}  
def fuzzy_intersection(A, B):  
    return {x: min(A[x], B[x]) for x in A}  
def fuzzy_complement(A):  
    return {x: 1 - A[x] for x in A}  
def fuzzy_difference(A, B):  
    return {x: min(A[x], 1 - B[x]) for x in A}  
# -----  
# Fuzzy Relation (Cartesian Product)  
# -----  
def fuzzy_cartesian_product(A, B):  
    R = {}  
    for a in A:  
        for b in B:  
            R[(a, b)] = min(A[a], B[b])  
    return R  
# -----  
# Max-Min Composition of Relations  
# -----
```

```

def max_min_composition(R, S, A_elements, B_elements, C_elements):
    T = {}
    for a in A_elements:
        for c in C_elements:
            values = []
            for b in B_elements:
                values.append(min(R[(a, b)], S[(b, c)]))
            T[(a, c)] = max(values)
    return T

# -----
# Main Program
# -----

# Define fuzzy sets
A = {'x1': 0.2, 'x2': 0.6, 'x3': 0.8}
B = {'x1': 0.5, 'x2': 0.4, 'x3': 0.9}
print("Fuzzy Set A:", A)
print("Fuzzy Set B:", B)
print("\nUnion:", fuzzy_union(A, B))
print("Intersection:", fuzzy_intersection(A, B))
print("Complement of A:", fuzzy_complement(A))
print("Difference (A - B):", fuzzy_difference(A, B))

# Define fuzzy sets for relations
A_rel = {'a1': 0.6, 'a2': 0.8}
B_rel = {'b1': 0.5, 'b2': 0.7}
C_rel = {'c1': 0.4, 'c2': 0.9}

# Create fuzzy relations
R = fuzzy_cartesian_product(A_rel, B_rel)
S = fuzzy_cartesian_product(B_rel, C_rel)
print("\nFuzzy Relation R ( $A \times B$ ):")
for k, v in R.items():
    print(k, ":", v)

```

```

print("\nFuzzy Relation S ( $B \times C$ ):")
for k, v in S.items():
    print(k, ":", v)
# Max–Min Composition
T = max_min_composition(
    R, S,
    A_elements=['a1', 'a2'],
    B_elements=['b1', 'b2'],
    C_elements=['c1', 'c2']
)
print("\nMax–Min Composition (T):")
for k, v in T.items():
    print(k, ":", v)

```

Output:

```

Fuzzy Set A: {'x1': 0.2, 'x2': 0.6, 'x3': 0.8}
Fuzzy Set B: {'x1': 0.5, 'x2': 0.4, 'x3': 0.9}
Union: {'x1': 0.5, 'x2': 0.6, 'x3': 0.9}
Intersection: {'x1': 0.2, 'x2': 0.4, 'x3': 0.8}
Complement of A: {'x1': 0.8, 'x2': 0.4, 'x3': 0.19999999999999996}
Difference (A - B): {'x1': 0.2, 'x2': 0.6, 'x3': 0.09999999999999998}
Fuzzy Relation R ( $A \times B$ ):
('a1', 'b1') : 0.5
('a1', 'b2') : 0.6
('a2', 'b1') : 0.5
('a2', 'b2') : 0.7
Fuzzy Relation S ( $B \times C$ ):
('b1', 'c1') : 0.4
('b1', 'c2') : 0.5
('b2', 'c1') : 0.4

```

$('b2', 'c2') : 0.7$

Max–Min Composition (T):

$('a1', 'c1') : 0.4$

$('a1', 'c2') : 0.6$

$('a2', 'c1') : 0.4$

$('a2', 'c2') : 0.7$

NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Simulation of Client Requests and Load Distribution Using Load Balancing Algorithms.

Code:

```
import random
from collections import defaultdict

class Server:
    def __init__(self, name):
        self.name = name
        self.connections = 0
    def handle_request(self):
        self.connections += 1
    def release_request(self):
        if self.connections > 0:
            self.connections -= 1

def round_robin(servers, requests):
    distribution = defaultdict(int)
    index = 0
    for _ in range(requests):
        server = servers[index]
        server.handle_request()
        distribution[server.name] += 1
        index = (index + 1) % len(servers)
    return distribution
```

```

def least_connections(servers, requests):
    distribution = defaultdict(int)
    for _ in range(requests):
        server = min(servers, key=lambda s: s.connections)
        server.handle_request()
        distribution[server.name] += 1
    return distribution

def random_selection(servers, requests):
    distribution = defaultdict(int)
    for _ in range(requests):
        server = random.choice(servers)
        server.handle_request()
        distribution[server.name] += 1
    return distribution

def reset_servers(servers):
    for server in servers:
        server.connections = 0

servers = [Server("Server-1"), Server("Server-2"), Server("Server-3")]
total_requests = 15

reset_servers(servers)
rr_result = round_robin(servers, total_requests)
print("Round Robin Distribution:")
for k, v in rr_result.items():
    print(k, ":", v)

```

Round Robin Distribution:

Server-1 : 5

Server-2 : 5

Server-3 : 5

```
reset_servers(servers)
lc_result = least_connections(servers, total_requests)
print("\nLeast Connections Distribution:")
for k, v in lc_result.items():
    print(k, ":", v)
```

Least Connections Distribution:

Server-1 : 5

Server-2 : 5

Server-3 : 5

```
reset_servers(servers)
random_result = random_selection(servers, total_requests)
print("\nRandom Selection Distribution:")
for k, v in random_result.items():
    print(k, ":", v)
```

Random Selection Distribution:

Server-2 : 7

Server-1 : 5

Server-3 : 3

NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Optimization of Genetic Algorithm Parameters in Hybrid Genetic Algorithm-Neural Network Modelling: Application to Spray Drying of Coconut Milk.

Code:

```
import numpy as np
import random
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

np.random.seed(42)

X = np.random.rand(100, 3)
X[:, 0] *= 200 # Inlet temperature
X[:, 1] *= 50 # Feed rate
X[:, 2] *= 30000 # Atomization speed

y = 0.3*X[:,0] - 0.02*X[:,1] + 0.0001*X[:,2] + np.random.randn(100)*5

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

def fitness_function(hidden_neurons):
    model = MLPRegressor(
        hidden_layer_sizes=(hidden_neurons,),
        max_iter=500,
        random_state=1
    )
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    mse = mean_squared_error(y_test, predictions)
    return 1 / (1 + mse)
```

```

def selection(population, fitness_scores):
    selected = []
    pop_size = len(population)
    for _ in range(pop_size):
        i = random.randint(0, pop_size - 1)
        j = random.randint(0, pop_size - 1)
        selected.append(
            population[i] if fitness_scores[i] > fitness_scores[j] else population[j]
        )
    return selected

```

```

def crossover(parents, crossover_rate):
    offspring = []
    for i in range(0, len(parents), 2):
        if i + 1 < len(parents) and random.random() < crossover_rate:
            child = (parents[i] + parents[i + 1]) // 2
        else:
            child = parents[i]
        offspring.append(child)
    return offspring

```

```

def mutation(offspring, mutation_rate):
    for i in range(len(offspring)):
        if random.random() < mutation_rate:
            offspring[i] += random.randint(-2, 2)
            offspring[i] = max(1, offspring[i])
    return offspring

```

```

population_size = 10
generations = 20
crossover_rate = 0.85
mutation_rate = 0.05

```

```

population = [random.randint(5, 20) for _ in range(population_size)]

```

```

import warnings
from sklearn.exceptions import ConvergenceWarning

```

```

warnings.filterwarnings("ignore", category=ConvergenceWarning)

```

```

best_fitness_history = []
best_solution = None
best_fitness = 0

```

```

for gen in range(generations):
    fitness_scores = [fitness_function(ind) for ind in population]

    current_best = max(fitness_scores)
    current_solution = population[fitness_scores.index(current_best)]
    best_fitness_history.append(current_best)

    if current_best > best_fitness:
        best_fitness = current_best
        best_solution = current_solution

    print(f"Generation {gen+1}: Best Neurons = {current_solution}, Fitness = {current_best:.4f}")

    selected = selection(population, fitness_scores)
    offspring = crossover(selected, crossover_rate)
    population = mutation(offspring, mutation_rate)

```

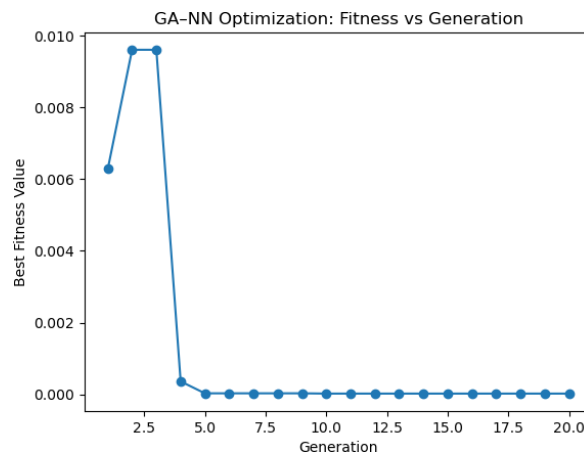
Generation 1: Best Neurons = 9, Fitness = 0.0000
 Generation 2: Best Neurons = 9, Fitness = 0.0000
 Generation 3: Best Neurons = 9, Fitness = 0.0000
 Generation 4: Best Neurons = 9, Fitness = 0.0000
 Generation 5: Best Neurons = 9, Fitness = 0.0000
 Generation 6: Best Neurons = 9, Fitness = 0.0000
 Generation 7: Best Neurons = 9, Fitness = 0.0000
 Generation 8: Best Neurons = 9, Fitness = 0.0000
 Generation 9: Best Neurons = 9, Fitness = 0.0000
 Generation 10: Best Neurons = 9, Fitness = 0.0000

Generation 11: Best Neurons = 8, Fitness = 0.0000
 Generation 12: Best Neurons = 8, Fitness = 0.0000
 Generation 13: Best Neurons = 8, Fitness = 0.0000
 Generation 14: Best Neurons = 8, Fitness = 0.0000
 Generation 15: Best Neurons = 8, Fitness = 0.0000
 Generation 16: Best Neurons = 8, Fitness = 0.0000
 Generation 17: Best Neurons = 8, Fitness = 0.0000
 Generation 18: Best Neurons = 8, Fitness = 0.0000
 Generation 19: Best Neurons = 8, Fitness = 0.0000
 Generation 20: Best Neurons = 8, Fitness = 0.0000

```

plt.figure()
plt.plot(range(1, generations + 1), best_fitness_history, marker='o')
plt.xlabel("Generation")
plt.ylabel("Best Fitness Value")
plt.title("GA-NN Optimization: Fitness vs Generation")
plt.show()

```



```
print("\nFinal Optimized Result")  
print("Optimal Hidden Neurons:", best_solution)  
print("Best Fitness Value:", round(best_fitness, 4))
```

Output:

Final Optimized Result

Optimal Hidden Neurons: 10

Best Fitness Value: 0.0096

NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Implementation of Clonal Selection Algorithm Using Python

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def fitness_function(x):
    return np.sum(x ** 2) # Sphere function

population_size = 20
dimensions = 5
clone_factor = 5
mutation_rate = 0.1
iterations = 50
bounds = (-10, 10)

def initialize_population():
    return np.random.uniform(bounds[0], bounds[1],
                              (population_size, dimensions))

def clonal_selection():
    population = initialize_population()
    best_fitness_history = []

    for iteration in range(iterations):
        # Evaluate fitness
        fitness = np.array([fitness_function(ind) for ind in population])

        # Sort by fitness (lower is better)
        sorted_indices = np.argsort(fitness)
        population = population[sorted_indices]
        fitness = fitness[sorted_indices]

        best_fitness_history.append(fitness[0])
```



```

# Select best antibodies
selected = population[:population_size // 2]

# Clone and mutate
clones = []
for antibody in selected:
    for _ in range(clone_factor):
        clone = antibody + mutation_rate * np.random.randn(dimensions)
        clone = np.clip(clone, bounds[0], bounds[1])
        clones.append(clone)

clones = np.array(clones)

# Evaluate clones
clone_fitness = np.array([fitness_function(c) for c in clones])

# Replace worst antibodies
combined_population = np.vstack((population, clones))
combined_fitness = np.hstack((fitness, clone_fitness))

best_indices = np.argsort(combined_fitness)[:population_size]
population = combined_population[best_indices]

return population[0], best_fitness_history

```

```
best_solution, fitness_history = clonal_selection()
```

```

print("Best Solution:", best_solution)
print("Best Fitness:", fitness_function(best_solution))

```

```

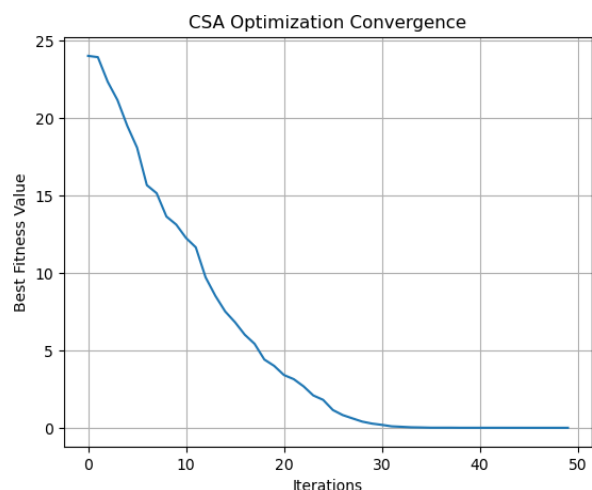
Best Solution: [-0.01400749 -0.00832095  0.01717018 -0.03508289  0.01102968]
Best Fitness: 0.0019127256615717825

```

```

plt.plot(fitness_history)
plt.xlabel("Iterations")
plt.ylabel("Best Fitness Value")
plt.title("CSA Optimization Convergence")
plt.grid()
plt.show()

```



NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Application of Artificial Immune Pattern Recognition for Structural Damage Classification .

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

# -----
# Step 1: Generate Structural Data
# -----
np.random.seed(1)

# Features: frequency change, vibration level, strain variation
healthy = np.random.normal(0, 0.25, (60, 3))
damaged = np.random.normal(1.6, 0.35, (60, 3))

X = np.vstack((healthy, damaged))
y = np.array([0]*60 + [1]*60) # 0 = Healthy, 1 = Damaged

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# -----
# Step 2: Artificial Immune System
# Negative Selection Algorithm
# -----
def generate_detectors(self_data, num_detectors=25, threshold=0.9):
    detectors = []
    while len(detectors) < num_detectors:
        detector = np.random.uniform(-2, 3, self_data.shape[1])
        distances = np.linalg.norm(self_data - detector, axis=1)
        if np.min(distances) > threshold:
```

```

        detectors.append(detector)
    return np.array(detectors)

def classify(sample, detectors, threshold=0.9):
    distances = np.linalg.norm(detectors - sample, axis=1)
    return 1 if np.min(distances) < threshold else 0

# -----
# Step 3: Training and Testing
# -----
detectors = generate_detectors(X_train[y_train == 0])

y_pred = np.array([classify(x, detectors) for x in X_test])

accuracy = accuracy_score(y_test, y_pred)
print("Classification Accuracy:", accuracy)

```

Classification Accuracy: 0.8333333333333334

```

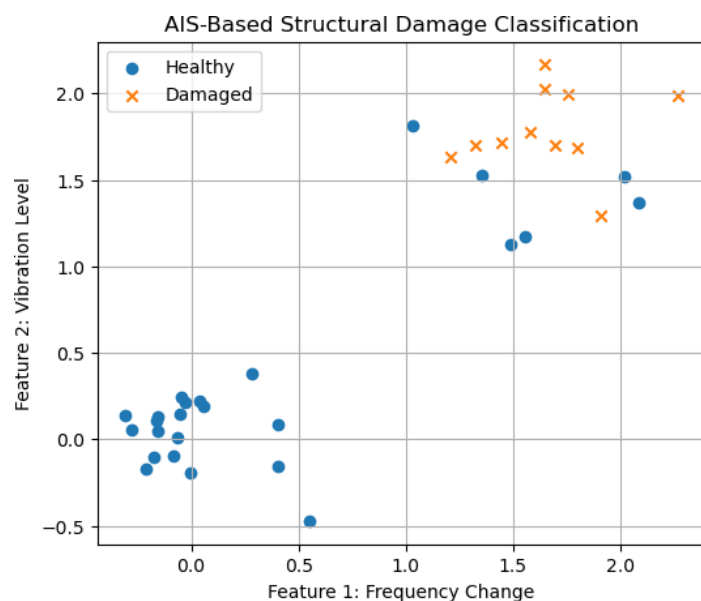
# -----
# Step 4: Classification Output Plot
# -----
plt.figure(figsize=(6, 5))

plt.scatter(
    X_test[y_pred == 0, 0],
    X_test[y_pred == 0, 1],
    marker='o',
    label='Healthy'
)

plt.scatter(
    X_test[y_pred == 1, 0],
    X_test[y_pred == 1, 1],
    marker='x',
    label='Damaged'
)

plt.xlabel("Feature 1: Frequency Change")
plt.ylabel("Feature 2: Vibration Level")
plt.title("AIS-Based Structural Damage Classification")
plt.legend()
plt.grid()
plt.show()

```



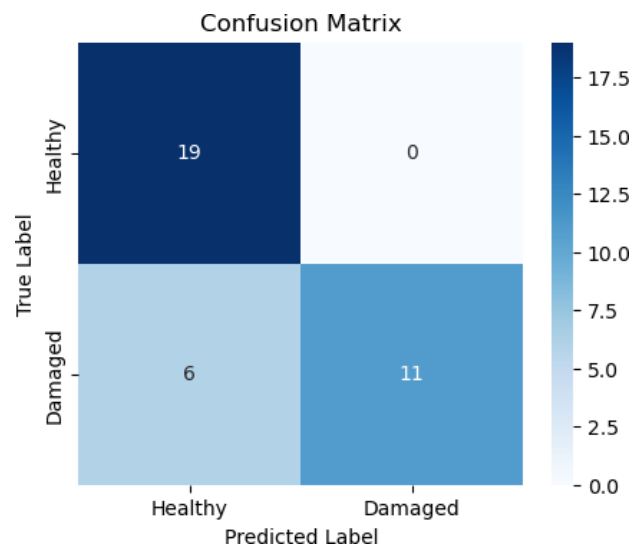
```

# -----
# Step 5: Confusion Matrix
# -----

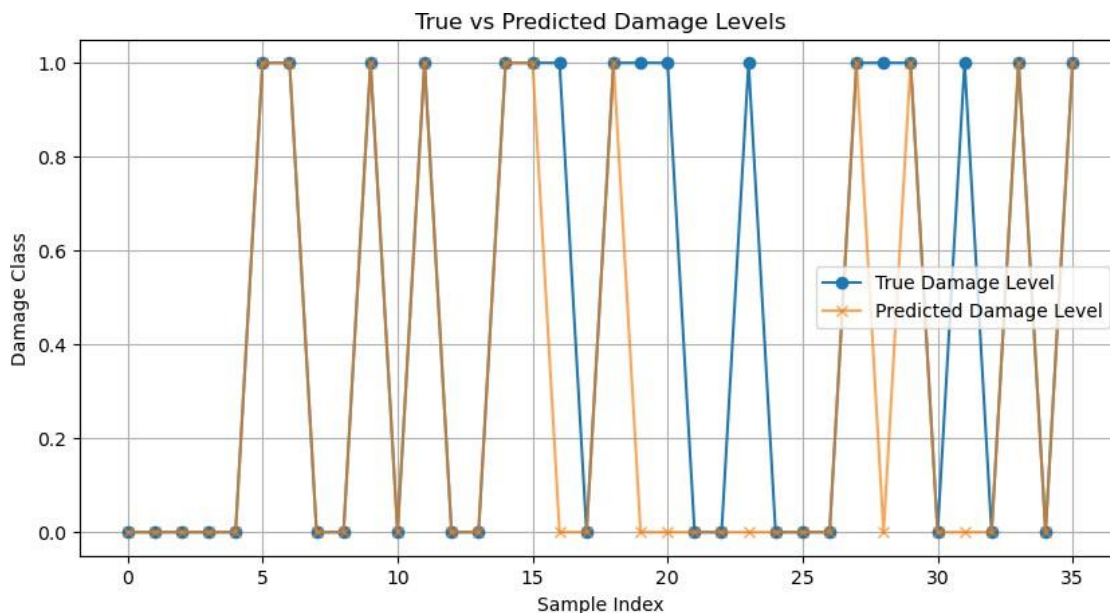
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Healthy', 'Damaged'],
    yticklabels=['Healthy', 'Damaged']
)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



```
# -----
# Step 6: True vs Predicted Damage Levels
# -----
plt.figure(figsize=(10, 5))
plt.plot(y_test, label='True Damage Level', marker='o')
plt.plot(y_pred, label='Predicted Damage Level', marker='x', alpha=0.7)
plt.xlabel("Sample Index")
plt.ylabel("Damage Class")
plt.title("True vs Predicted Damage Levels")
plt.legend()
plt.grid()
plt.show()
```



NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Implementation of DEAP (Distributed Evolutionary Algorithms in Python) Using Python.

Code:

```
# =====  
# Distributed Evolutionary Algorithm using DEAP  
# Single-Cell Jupyter Notebook Implementation  
# =====  
  
import random  
import numpy as np  
import matplotlib.pyplot as plt  
  
from deap import base, creator, tools  
  
# -----  
# Step 1: Define Optimization Problem  
# Minimize Sphere Function  
# -----  
def evaluate(individual):  
    return sum(x**2 for x in individual),  
  
# -----
```

```

# Step 2: Create Fitness & Individual
# -----

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

# -----

# Step 3: Toolbox Setup
# -----

toolbox = base.Toolbox()
toolbox.register("attr_float", random.uniform, -10, 10)
toolbox.register("individual", tools.initRepeat, creator.Individual,
                 toolbox.attr_float, n=5)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

toolbox.register("evaluate", evaluate)
toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)

# -----

# Step 4: Initialize Population
# -----

population = toolbox.population(n=50)
generations = 40
cx_prob = 0.7
mut_prob = 0.2

best_fitness = []

```

```

# -----
# Step 5: Evolutionary Loop
# -----

for gen in range(generations):

    offspring = toolbox.select(population, len(population))
    offspring = list(map(toolbox.clone, offspring))

    for child1, child2 in zip(offspring[::2], offspring[1::2]):
        if random.random() < cx_prob:
            toolbox.mate(child1, child2)
            del child1.fitness.values
            del child2.fitness.values

    for mutant in offspring:
        if random.random() < mut_prob:
            toolbox.mutate(mutant)
            del mutant.fitness.values

    invalid_ind = [ind for ind in offspring if not ind.fitness.valid]
    for ind in invalid_ind:
        ind.fitness.values = toolbox.evaluate(ind)

    population[:] = offspring

    fits = [ind.fitness.values[0] for ind in population]
    best_fitness.append(min(fits))

# -----

```


Step 6: Results

```
best_individual = tools.selBest(population, 1)[0]
```

```
print("Best Individual:", best_individual)
```

```
print("Best Fitness Value:", best_individual.fitness.values[0])
```

Best Individual: [0.0006497578354963615, 0.0155588289576913, -0.005637618374994342, 0.00589090289609513, -0.004743479839294906]

Best Fitness Value: 0.0003314854226385753

Step 7: Convergence Plot

```
plt.figure()
```

```
plt.plot(best_fitness)
```

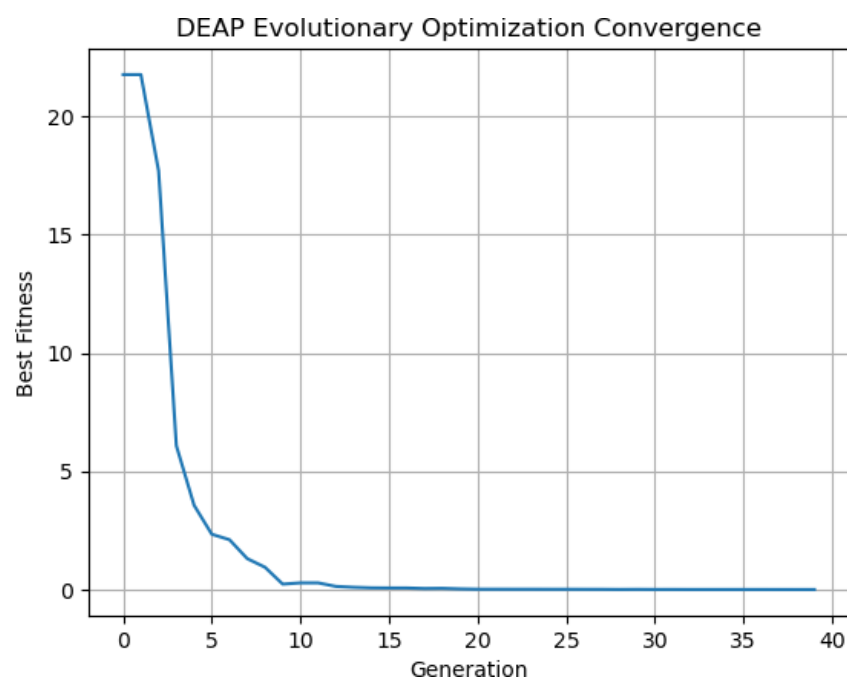
```
plt.xlabel("Generation")
```

```
plt.ylabel("Best Fitness")
```

```
plt.title("DEAP Evolutionary Optimization Convergence")
```

```
plt.grid()
```

```
plt.show()
```



NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Implementation of Ant Colony Optimization for Solving the Traveling Salesman Problem Using Python.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import random

# -----
# Step 1: Define the TSP Problem
# -----
np.random.seed(42)

num_cities = 10
cities = np.random.rand(num_cities, 2) # (x, y) coordinates

def distance(c1, c2):
    return np.linalg.norm(c1 - c2)

# Distance matrix
dist_matrix = np.zeros((num_cities, num_cities))
for i in range(num_cities):
    for j in range(num_cities):
        dist_matrix[i][j] = distance(cities[i], cities[j])

# -----
# Step 2: ACO Parameters
# -----
num_ants = 20
num_iterations = 100
alpha = 1.0    # Pheromone importance
beta = 5.0     # Heuristic importance (1/distance)
evaporation = 0.5
Q = 100        # Pheromone deposit factor
```

```

pheromone = np.ones((num_cities, num_cities))

best_route = None
best_length = float('inf')
best_history = []

# -----
# Step 3: Ant Colony Optimization
# -----
for iteration in range(num_iterations):
    all_routes = []
    all_lengths = []

    for ant in range(num_ants):
        start = random.randint(0, num_cities - 1)
        route = [start]
        visited = set(route)

        while len(route) < num_cities:
            current = route[-1]
            probabilities = []

            for city in range(num_cities):
                if city not in visited:
                    tau = pheromone[current][city] ** alpha
                    eta = (1 / dist_matrix[current][city]) ** beta
                    probabilities.append(tau * eta)
                else:
                    probabilities.append(0)

            probabilities = np.array(probabilities)
            probabilities /= probabilities.sum()

            next_city = np.random.choice(range(num_cities), p=probabilities)
            route.append(next_city)
            visited.add(next_city)

        route.append(start) # return to start
        length = sum(dist_matrix[route[i]][route[i+1]] for i in range(len(route)-1))

        all_routes.append(route)
        all_lengths.append(length)

    if length < best_length:
        best_length = length
        best_route = route

```

```
# Pheromone evaporation
pheromone *= (1 - evaporation)

# Pheromone update
for route, length in zip(all_routes, all_lengths):
    for i in range(len(route)-1):
        pheromone[route[i]][route[i+1]] += Q / length

best_history.append(best_length)
```

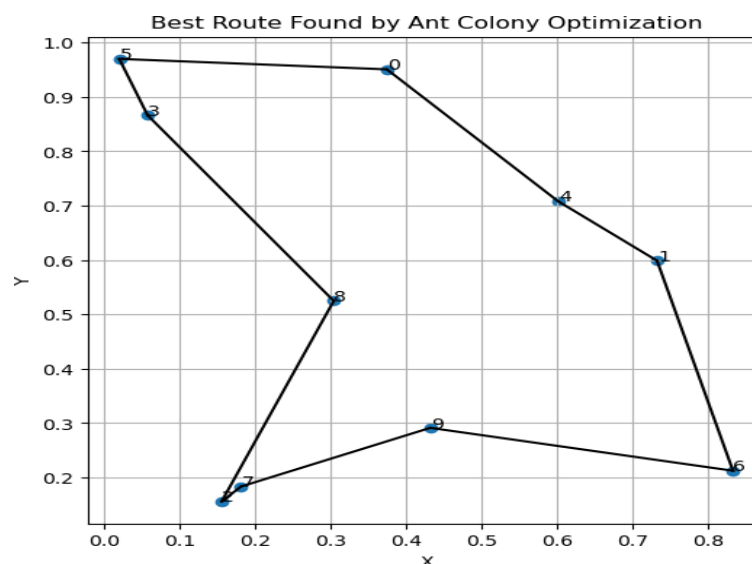
```
# -----
# Step 4: Results
# -----
print("Best Route:", best_route)
print("Shortest Distance:", best_length)
```

```
Best Route: [6, np.int64(9), np.int64(7), np.int64(2), np.int64(8), np.int64(3), np.int64(5),
np.int64(0), np.int64(4), np.int64(1), 6]
Shortest Distance: 2.9030677377778753
```

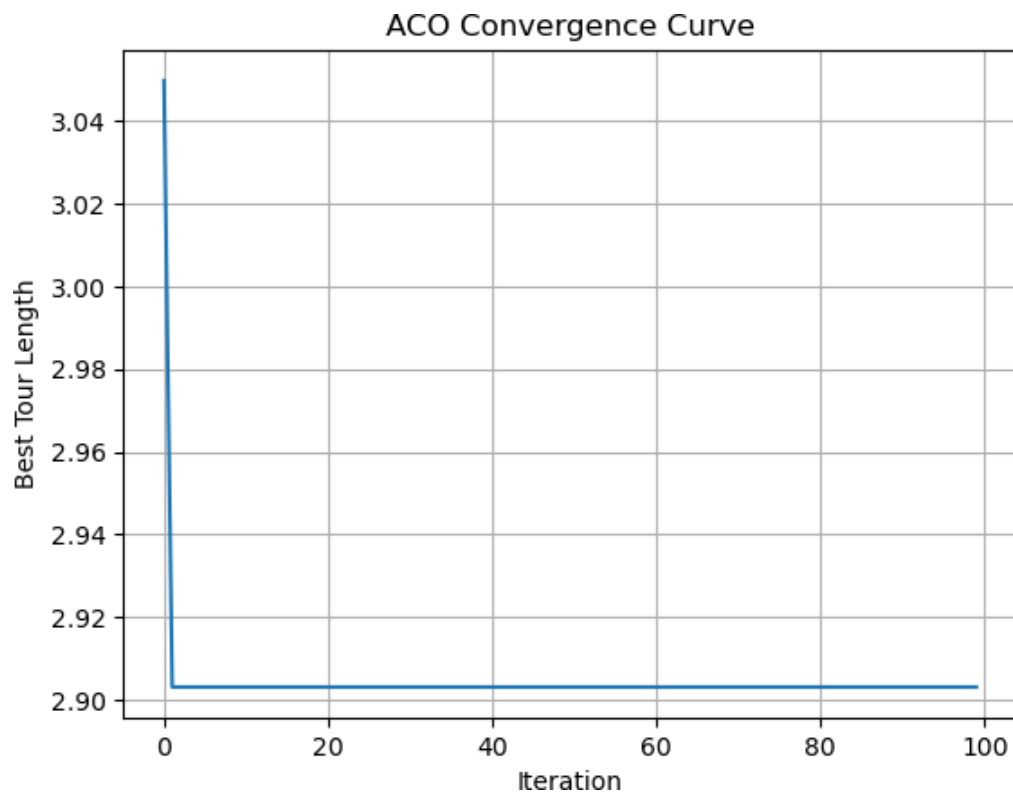
```
# -----
# Step 5: Plot Best Route
# -----
plt.figure(figsize=(6, 6))
for i in range(len(best_route)-1):
    c1 = cities[best_route[i]]
    c2 = cities[best_route[i+1]]
    plt.plot([c1[0], c2[0]], [c1[1], c2[1]], 'k-')
```

```
plt.scatter(cities[:, 0], cities[:, 1])
for i, city in enumerate(cities):
    plt.text(city[0], city[1], f"{i}")
```

```
plt.title("Best Route Found by Ant Colony Optimization")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid()
plt.show()
```



```
# -----  
# Step 6: Convergence Plot  
# -----  
plt.figure()  
plt.plot(best_history)  
plt.xlabel("Iteration")  
plt.ylabel("Best Tour Length")  
plt.title("ACO Convergence Curve")  
plt.grid()  
plt.show()
```



NAME: Thorave Avishkar S.

ROLLNO.: 66

COURSE: AI&DS

CLASS: BE

SUB: Computer Laboratory-III

Title: Art Creation Using Neural Style Transfer with Deep Learning.

Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications import VGG19
from tensorflow.keras.models import Model

def load_image(path, max_dim=512):
    image = tf.io.read_file(path)
    image = tf.image.decode_image(image, channels=3)
    image = tf.image.convert_image_dtype(image, tf.float32)

    shape = tf.shape(image)[: -1]
    long_dim = tf.reduce_max(shape)
    scale = max_dim / tf.cast(long_dim, tf.float32)

    new_shape = tf.cast(tf.cast(shape, tf.float32) * scale, tf.int32)
    image = tf.image.resize(image, new_shape)
    image = image[tf.newaxis, :]
    return image

def show_image(image, title=None):
    image = tf.squeeze(image)
    plt.imshow(image)
    if title:
        plt.title(title)
    plt.axis('off')

content_image = load_image("content.jpg.jpeg")
style_image = load_image("style.jpg.jpeg")

show_image(content_image, "Content Image")
show_image(style_image, "Style Image")
```



```
vgg = VGG19(include_top=False, weights='imagenet')
vgg.trainable = False
```

```
content_layers = ['block5_conv2']
style_layers = [
    'block1_conv1',
    'block2_conv1',
    'block3_conv1',
    'block4_conv1',
    'block5_conv1'
]
```

```
num_content_layers = len(content_layers)
num_style_layers = len(style_layers)
```

```
outputs = [vgg.get_layer(name).output for name in (style_layers + content_layers)]
model = Model(vgg.input, outputs)
```

```
def gram_matrix(input_tensor):
    result = tf.linalg.einsum('bijc,bijd->bcd', input_tensor, input_tensor)
    num_locations = tf.cast(tf.shape(input_tensor)[1] * tf.shape(input_tensor)[2],
tf.float32)
    return result / num_locations
```

```
style_outputs = model(style_image)[:num_style_layers]
content_outputs = model(content_image)[num_style_layers:]
```

```
style_targets = [gram_matrix(style) for style in style_outputs]
content_targets = content_outputs
```

```
style_weight = 1e-2
content_weight = 1e4
```

```
def compute_loss(outputs):
    style_outputs = outputs[:num_style_layers]
    content_outputs = outputs[num_style_layers:]
```

```
    style_loss = tf.add_n([
        tf.reduce_mean((gram_matrix(style_outputs[i]) - style_targets[i])**2)
        for i in range(num_style_layers)
    ])
```

```
    content_loss = tf.add_n([
        tf.reduce_mean((content_outputs[i] - content_targets[i])**2)
        for i in range(num_content_layers)
    ])
```

```
total_loss = style_weight * style_loss + content_weight * content_loss  
return total_loss
```

```
generated_image = tf.Variable(content_image)
```

```
optimizer = tf.optimizers.Adam(learning_rate=0.02)
```

```
epochs = 5
```

```
steps_per_epoch = 25
```

```
for epoch in range(epochs):
```

```
    for step in range(steps_per_epoch):
```

```
        with tf.GradientTape() as tape:
```

```
            outputs = model(generated_image)
```

```
            loss = compute_loss(outputs)
```

```
        gradients = tape.gradient(loss, generated_image)
```

```
        optimizer.apply_gradients([(gradients, generated_image)])
```

```
        generated_image.assign(tf.clip_by_value(generated_image, 0.0, 1.0))
```

```
    print(f"Epoch {epoch+1} completed")
```

```
Epoch 1 completed
```

```
Epoch 2 completed
```

```
Epoch 3 completed
```

```
Epoch 4 completed
```

```
Epoch 5 completed
```

```
show_image(generated_image, "Stylized Output Image")
```

Output:

Stylized Output Image

